

Resume Complet des 5 Agents IA

Pour chaque agent : ce qu'il fait, son flux d'execution, ses fonctions Python en detail, et ses bibliotheques.

37. WebResearch-Firecrawl

[37-WebResearch-Firecrawl/research_agent.py](#)

FONCTIONNALITE

Assistant de recherche web de type RAG (Retrieve-then-Generate). L'utilisateur pose une question ; l'outil interroge le web via `firecrawl.search()` qui renvoie a la fois les resultats ET leur contenu scrape en markdown, en un seul appel reseau. Ce contenu est ensuite transmis a un agent Groq charge uniquement de REDIGER une synthese sourcee : chaque affirmation doit citer sa source [1][2]..., et le modele doit signaler si les sources sont insuffisantes plutot que d'inventer une reponse.

FLUX D'EXECUTION

```
research() -> search_web() -> [ _extract_source() x N ] ->
synthesis_agent.run()
```

FONCTIONS

<code>search_web(query, limit)</code>	Parametres : la requete et le nombre max de resultats. Appelle <code>firecrawl.search()</code> , filtre les resultats invalides, renvoie une liste de Source.
<code>_extract_source(item)</code>	Convertit UN resultat brut Firecrawl (objet Document) en Source (titre, url, contenu). Renvoie None si la page a echoue au scraping (site bloque, JS-only...).
<code>research(query, limit)</code>	Fonction principale appelee par l'interface. Enchaîne recherche puis appel LLM ; renvoie un tuple (rapport_texte, liste_sources).

BIBLIOTHEQUES

<code>streamlit</code>	interface web (formulaire, affichage du rapport et des sources)
<code>agno</code>	framework d'agent IA (classe Agent, gestion du prompt et du modele)
<code>firecrawl-py</code>	recherche web + scraping de contenu en un appel
<code>groq</code>	fournisseur du LLM de synthese (openai/gpt-oss-120b)
<code>python-dotenv</code>	charge les cle API depuis le fichier .env
<code>pydantic</code>	structure et valide les donnees (classe Source)

38. SiteMap-Firecrawl

[38-SiteMap-Firecrawl/map_agent.py](#)

FONCTIONNALITE

Outil de cartographie de site. Utilise firecrawl.map(), l'appel Firecrawl le moins couteux car il decouvre uniquement la LISTE des URLs d'un site (via son sitemap et ses liens internes) SANS scraper le contenu de chaque page. La liste brute est ensuite envoyee a un agent Groq qui la regroupe en categories coherentes (Documentation, Blog, API, Tarifs...), en reprenant les URLs telles quelles pour eviter toute hallucination de lien.

FLUX D'EXECUTION

```
map_site() -> explore_site() -> categorize() -> liste de Category
```

FONCTIONS

<code>explore_site(url, search, limit)</code>	Parametres : URL du site, filtre mot-cle optionnel, nombre max d'URLs. Appelle firecrawl.map(), renvoie une liste de Link (url, titre).
<code>categorize(links)</code>	Envoie la liste de liens au LLM avec consigne stricte de ne pas inventer d'URL. Renvoie une liste de Category (nom + urls).
<code>map_site(url, search, limit)</code>	Fonction principale : explore puis categorise. Renvoie un tuple (liens_bruts, categories).

BIBLIOTHEQUES

<code>streamlit</code>	interface web (slider de limite, affichage en grille de categories)
<code>agno</code>	framework d'agent IA, sortie structuree via output_schema
<code>firecrawl-py</code>	decouverte d'URLs d'un site (map)
<code>groq</code>	LLM de categorisation (openai/gpt-oss-120b)
<code>python-dotenv</code>	charge les cle API depuis le fichier .env
<code>pydantic</code>	sortie structuree validee (Category, SiteMapResult)

39. SiteCrawl-Firecrawl

[39-SiteCrawl-Firecrawl/crawl_agent.py](#)

FONCTIONNALITE

Outil d'exploration multi-pages. Utilise firecrawl.crawl(), qui part d'une URL de depart, suit ses liens internes et SCRAPE chaque page visitee (contrairement a map() qui ne fait que lister). C'est l'appel Firecrawl le plus couteux en credits, la limite de pages est donc volontairement basse par default. Le contenu de toutes les pages visitees est ensuite envoye a un agent Groq qui produit une fiche de synthese du site : vue d'ensemble, themes principaux, pages notables, chaque point etant cite [1][2]...

FLUX D'EXECUTION

```
digest() -> crawl_site() -> digest_agent.run()
```

FONCTIONS

crawl_site(url, limit)	Parametres : URL de depart, nombre max de pages a visiter. Appelle firecrawl.crawl(), renvoie une liste de Page (titre, url, contenu).
digest(url, limit)	Fonction principale : crawl le site puis demande la fiche de synthese au LLM. Renvoie un tuple (fiche_texte, pages_visitees).

BIBLIOTHEQUES

streamlit	interface web (slider de pages, affichage des pages explorees)
agno	framework d'agent IA
firecrawl-py	exploration et scraping multi-pages (crawl)
groq	LLM de synthese (openai/gpt-oss-120b)
python-dotenv	charge les cles API depuis le fichier .env
pydantic	structure de donnees (classe Page)

40. PageAnalyzer-Firecrawl

[40-PageAnalyzer-Firecrawl/scrape_agent.py](#)

FONCTIONNALITE

Outil d'analyse d'UNE seule page web. Utilise firecrawl.scrape(), l'appel Firecrawl le plus simple : il lit une URL precise et renvoie son contenu nettoye en markdown. Ce contenu est transmis a un agent Groq qui repond a une question precise de l'utilisateur sur cette page, ou en produit un resume structure si aucune question n'est posee. Contrairement aux 4 autres agents, celui-ci ne traite qu'une seule page a la fois : le contenu garde peut donc etre plus long (12 000 caracteres) sans risquer de depasser le quota du modele.

FLUX D'EXECUTION

```
analyze_page() -> scrape_page() -> page_agent.run()
```

FONCTIONS

scrape_page(url)	Appelle firecrawl.scrape(), renvoie le contenu markdown de la page, ou None si l'extraction a echoue.
-------------------------	---

**analyze_page(url,
question)**

Fonction principale. Scrape la page puis, selon que 'question' est fournie ou non, demande au LLM une reponse ciblee ou un resume general. Renvoie (reponse, erreur).

BIBLIOTHEQUES

streamlit	interface web (champ URL + champ question optionnel)
agno	framework d'agent IA
firecrawl-py	lecture du contenu d'une page (scrape)
groq	LLM de question-reponse et de resume (openai/gpt-oss-120b)
python-dotenv	charge les cle API depuis le fichier .env

41. DocumentCompare-MistralOCR

41-DocumentCompare-MistralOCR/compare_agent.py

FONCTIONNALITE

Compareur de deux versions d'un document PDF (ex : deux versions d'un contrat). Chaque PDF est d'abord transforme en texte via l'OCR de Mistral (mistral-ocr-latest). Les deux textes sont ensuite envoyes a un agent Mistral (mistral-large-latest, plus puissant que Groq pour une tache d'analyse fine) qui identifie les differences de FOND entre les deux versions -- Ajout, Suppression, Modification -- en ignorant le bruit purement cosmetique (mise en page, artefacts d'OCR). La sortie est structuree (JSON valide via Pydantic) pour permettre un affichage avec badges colores par type de changement.

FLUX D'EXECUTION

```
compare_pdfs() -> ocr_pdf() x2 -> compare_documents() ->
comparison_agent.run()
```

FONCTIONS

<code>ocr_pdf(file_path)</code>	Upload le fichier PDF vers Mistral, recupere une URL signee, lance l'OCR, renvoie le texte markdown de toutes les pages.
<code>compare_documents(text_a, text_b)</code>	Envoie les deux textes au LLM Mistral avec consigne de ne signaler que des changements factuels justifiables. Renvoie un ComparisonResult (resume + liste de Change).
<code>compare_pdfs(path_a, path_b)</code>	Fonction principale : OCR des deux fichiers puis comparaison. Renvoie (resultat, texte_a, texte_b).

BIBLIOTHEQUES

<code>streamlit</code>	interface web (deux zones d'upload cote a cote)
<code>agno</code>	framework d'agent IA, sortie structuree via output_schema
<code>mistralai</code>	client Mistral : OCR (mistral-ocr-latest) et LLM (mistral-large-latest)
<code>python-dotenv</code>	charge la cle API Mistral depuis le fichier .env
<code>pydantic</code>	sortie structuree validee (Change, ComparisonResult)
